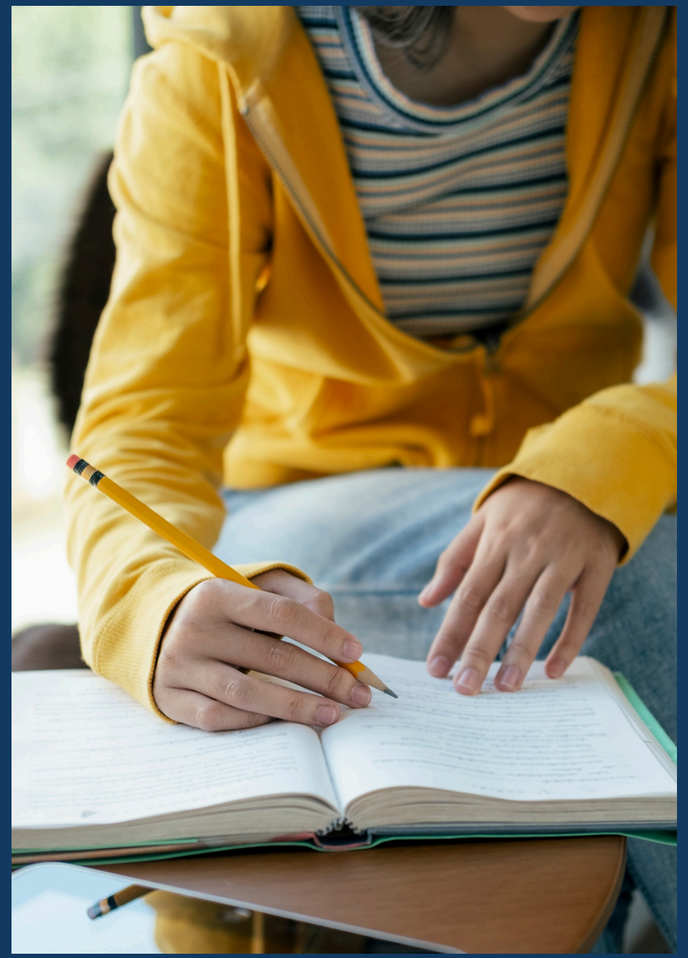
C. KANDASWAMI NAIDU COLLEGE

STUDY BUDDY

“ITS BETTER TO STUDDY TOGETHER”



Secure APIs . MongoDB

Presented by: III BSc

PROJECT OVERVIEW

A secure AI learning companion

One backend, several high-value study workflows.

What it does

AI StudyBuddy helps students upload study materials, generate concise summaries, create flashcards and quizzes, and receive personalized study plans.

Technology foundation

Node.js · Express.js
MongoDB · Mongoose ODM
JWT · RBAC · bcryptjs
Google Gemini AI

Project Team Members

Name	Email ID	Roll Number
Avinash R	avianash007@gmail.com	222404709
Jaisurya	jaisuryase@gmail.com	222404716
Dharshan	dharshannnnnn29@gmail.com	222404713
Santhosh kumar	santhoskumarsolan@gmail.com	222404730

SYSTEM REQUIREMENTS

A lightweight stack for dependable development

Software requirements

- Windows 10/11, macOS or Linux
- Node.js v16+
- npm v8+
- Express.js
- MongoDB
- Postman or Thunder Client
- Visual Studio Code

Hardware requirements

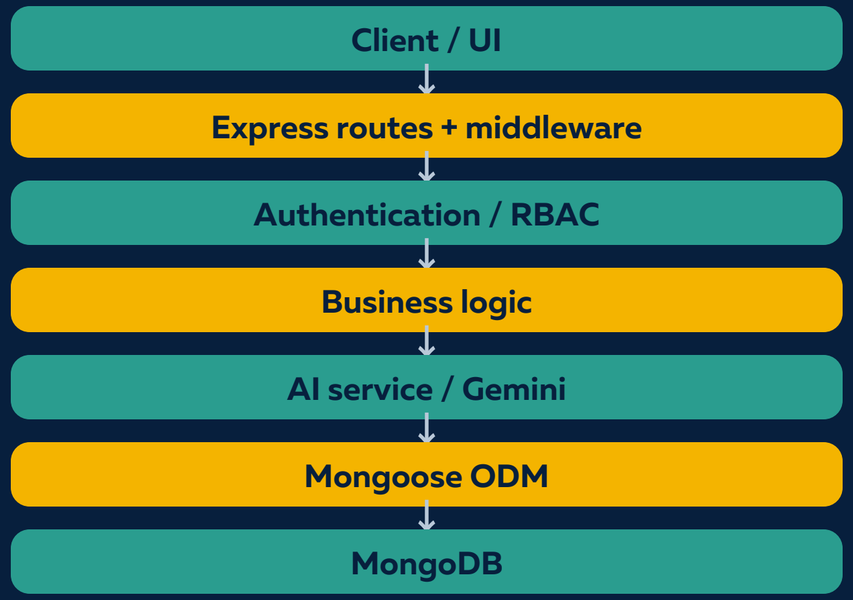
- Intel Core i5 8th Gen+ / AMD Ryzen 5 equivalent
- 8 GB RAM minimum; 16 GB recommended
- 1 GB available storage

Designed for a practical student development environment: local APIs, database inspection and iterative AI testing.

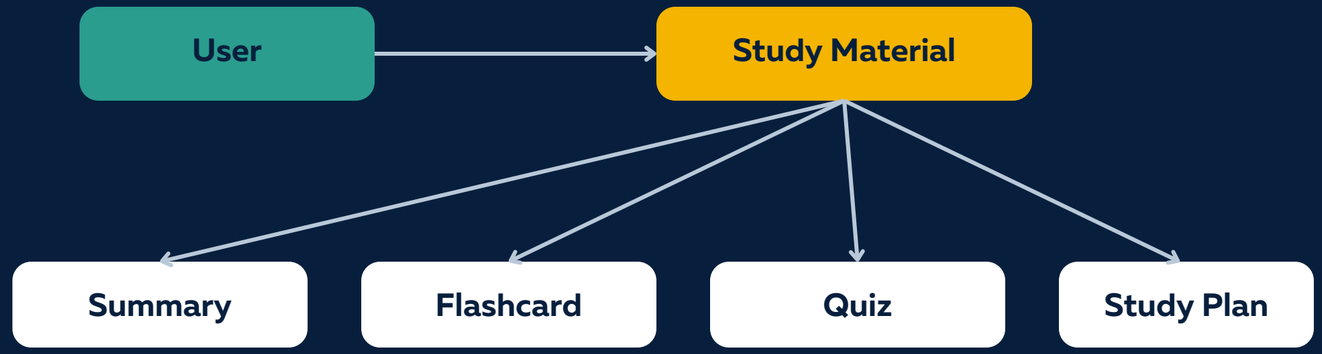
EPIC 1

Project Architecture & ER Diagram

Modular REST APIs separate access control, business logic, AI processing and persistence.



Logical data relationships



User uploads Study Material; generated resources are stored for retrieval and continuous learning.

EPIC 2

Project Setup and Configuration

A repeatable setup sequence keeps the backend easy to onboard and run.

01 **Create folder**
AI-STUDYBUDDY → open in VS Code

02 **Initialize**
npm init -y

03 **Install runtime**
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai

04 **Install dev tool**
npm install --save-dev nodemon

Environment configuration

```
PORT=5000
MONGO_URI=<your MongoDB connection>
JWT_SECRET=<your secret key>
GEMINI_API_KEY=<your Gemini key>
```

Keep secrets in .env and out of source control.

EPIC 3

Backend Development & Operational

Evidence panels are intentionally labeled; insert exact operational screenshots where available.

Execution

01 Environment configuration & crash troubleshooting

Placeholder — add exact .env / startup evidence

02 Port conflict diagnosis

Command: netstat -ano | findstr :5000

03 Incorrect endpoint routing

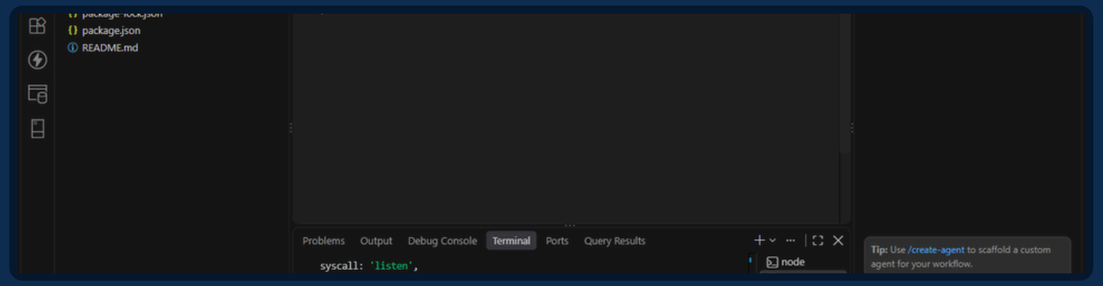
/apiuthuser/register → 404 Not Found

04 Project directory structure

package.json · package-lock.json · .env · index.js
README · src · uploads · node_modules

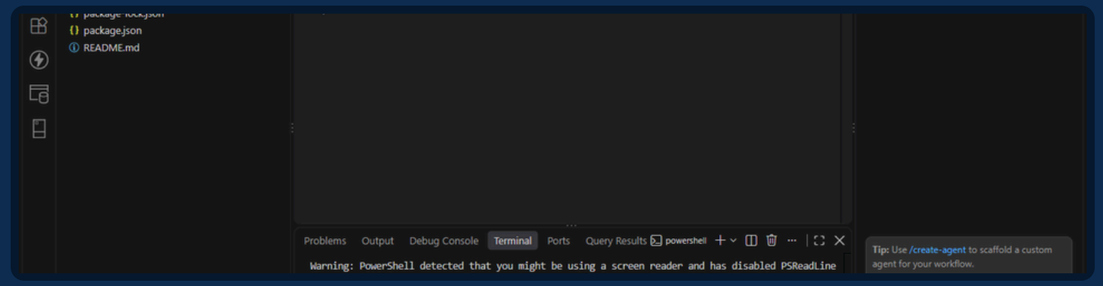
Operational walkthrough

Configuration, diagnostics, routing validation, and project organization.



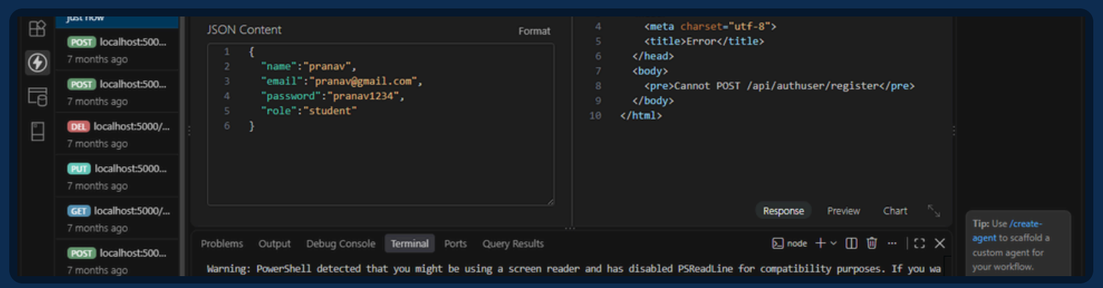
Environment Configuration & Server Crash Troubleshooting

The .env file defines PORT=5000, MongoDB, JWT, Gemini, and development settings; nodemon reports the app crash and waits for file changes.



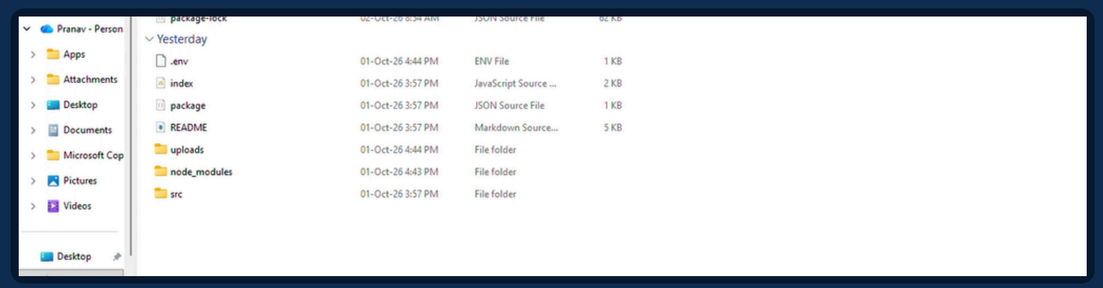
Port Conflict Diagnosis

The command identifies the process occupying port 5000 so the conflict can be resolved before restarting the server.



Incorrect Endpoint Routing Validation

The unmapped route confirms that Express rejects misspelled or unsupported endpoints.



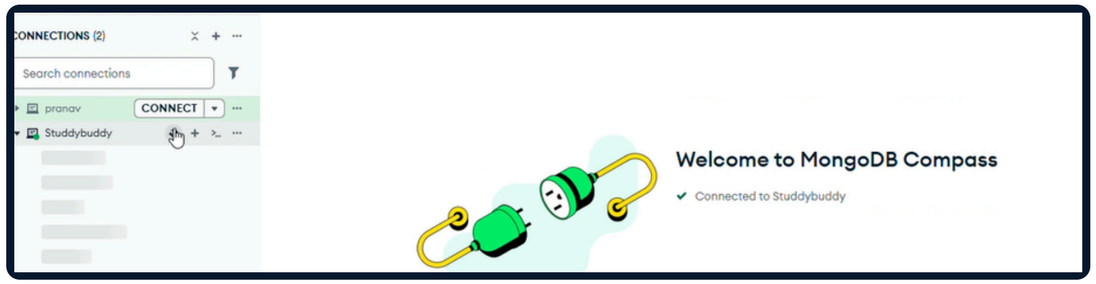
Project Directory Structure

The project is organized into configuration files, the server entry point, source modules, uploads, and installed dependencies.

DATABASE & API EVIDENCE

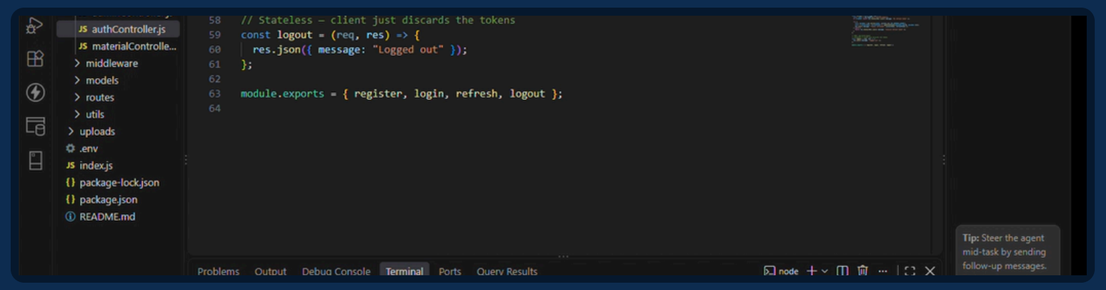
Verification checkpoints

Database connectivity, dependency setup, server readiness, and registration response.



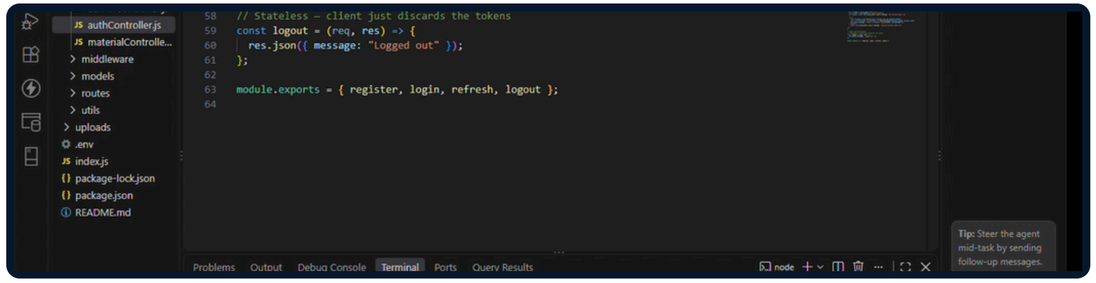
MongoDB Compass Connection

MongoDB Compass provides a visual check of the local database connection and stored collections.



Installing Project Dependencies

All packages listed in package.json are installed before application launch.



Starting the Server with Nodemon

Nodemon starts the development server and confirms environment, database, and port readiness.



Successful User Registration API Response

The response confirms password hashing, database insertion, token generation, and successful registration.